Aula 11 – Ordenação

Algortimos Avançados – Quick Sort

Pesquisa Sequencial, Binária, Probabilistica

Quick Sort

- Fazer o mesmo que o MergeSort sem utilizar o array auxiliar
- Algoritmo
- Dividir o vector em três sub vectores
  - 1º vector – Um elemento pivot
  - 2º vector – todos os elementos que sejam maiores que o pivot
  - 3º vector- todos os elementos que sejam menores que o pivot
- Repetir o processo para o 2º e 3º vector

Quick Sort

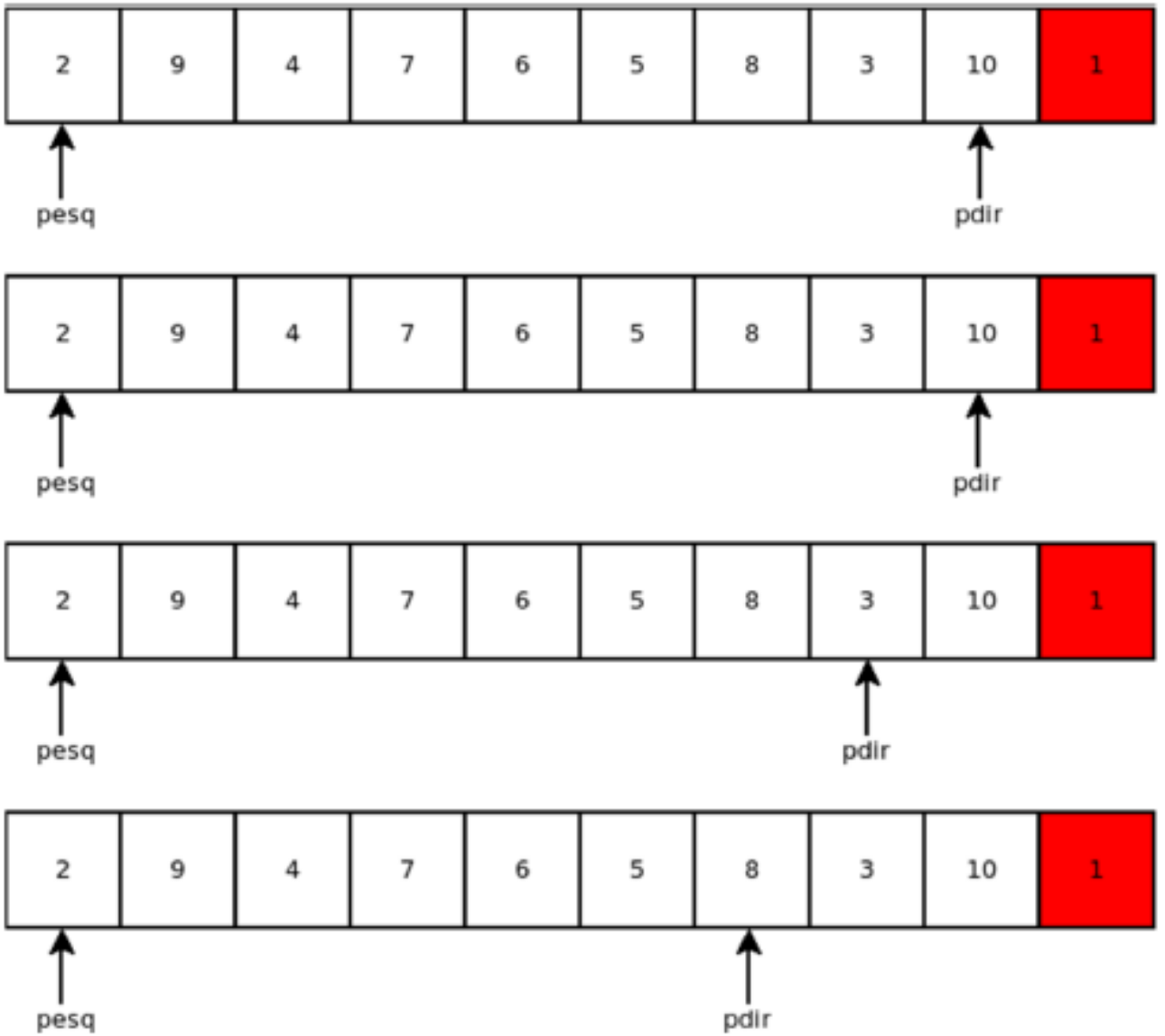
```
Object[] lista = ..... //Lista a ordenar
quickSort(lista, 0, length-1);


void quickSort(Object[] lista, int esquerda, int direita) {
        Se  direita > esquerda
            pivot = particao(esquerda, direita)
            quickSort(lista, esquerda, pivot-1)
            quickSort(lista, pivot + 1, direita)
 }
```

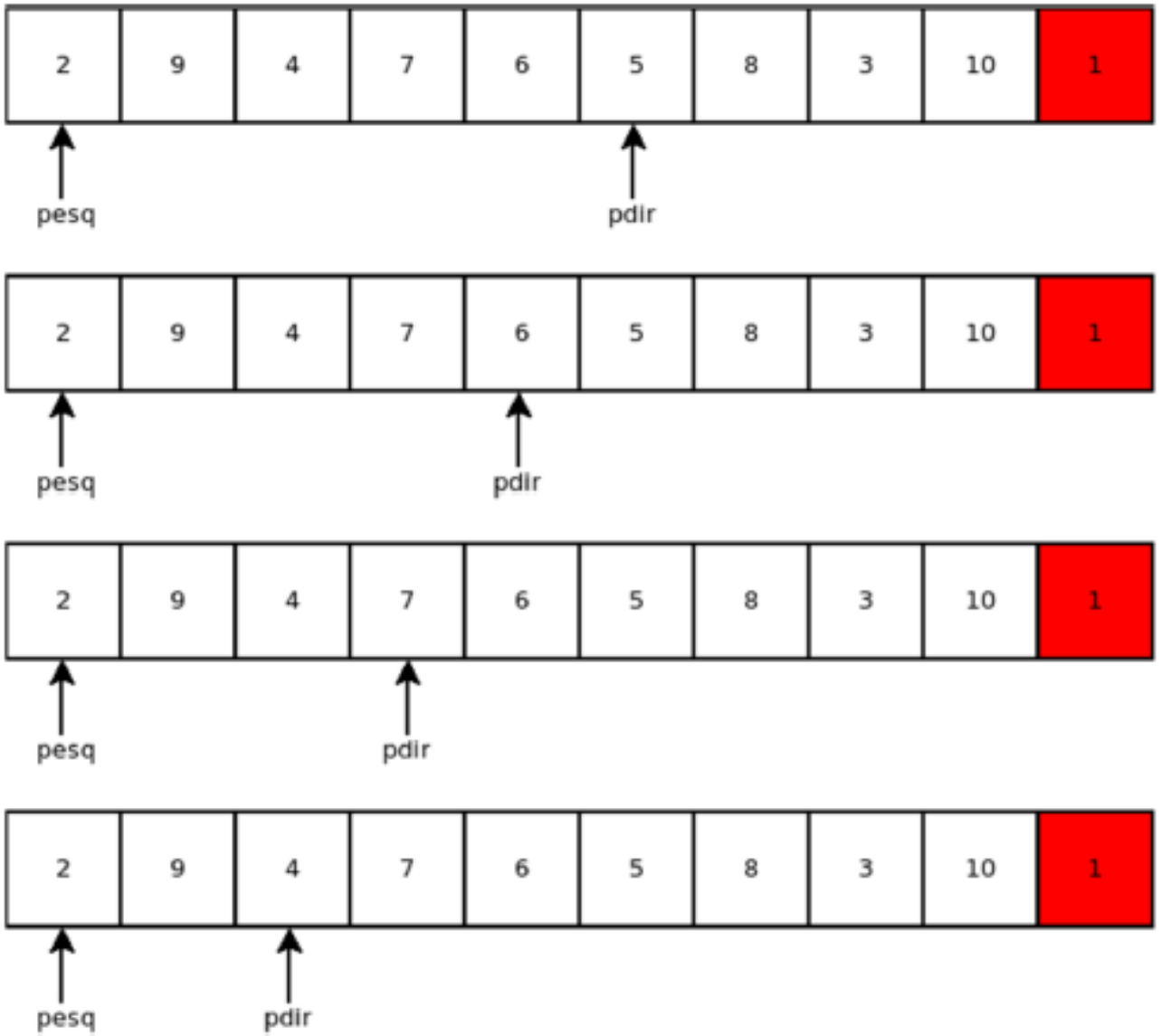
Quick Sort

```
int particao (Object[] lista, int esquerda, int direita) {
    pivot = lista[direita]
    pesq = esquerda
    pdir = direita -1
    acabou = falso
    enquanto não acabou
        enquanto lista[pesq] < pivot
            pesq = pesq + 1
        enquanto lista[pdir] > pivot
            pdir = pdir - 1
        se pesq > pdir
            acabou = verdadeiro
        se não
            troca(lista, pesq, pdir)
            pesq = pesq + 1
            pdir = pdir – 1
            Se pesq > pdir
                acabou = verdadeiro
    troca (lista, pesq, direita)
    devolve pesq
 }
```

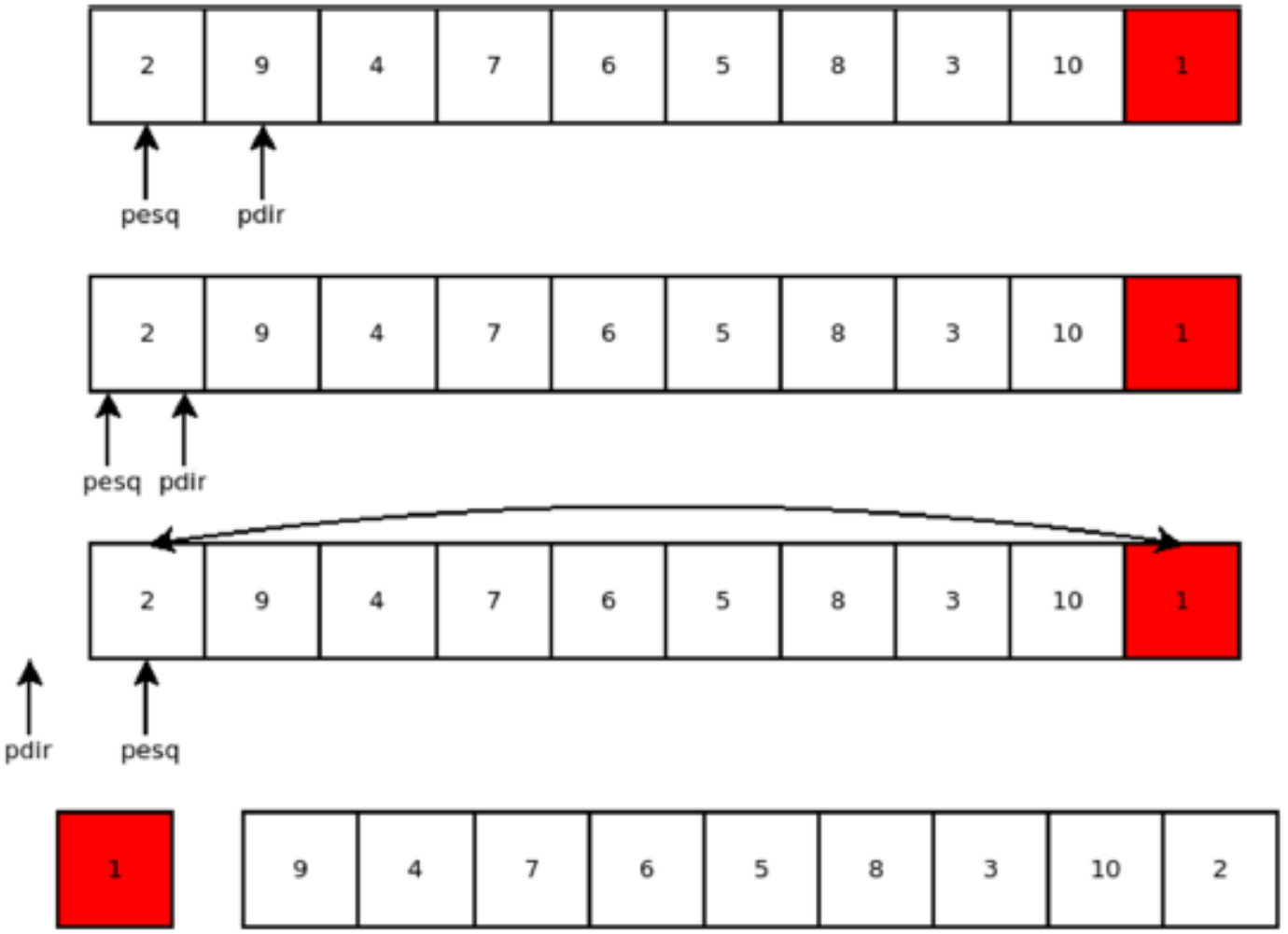
Quick Sort


2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir

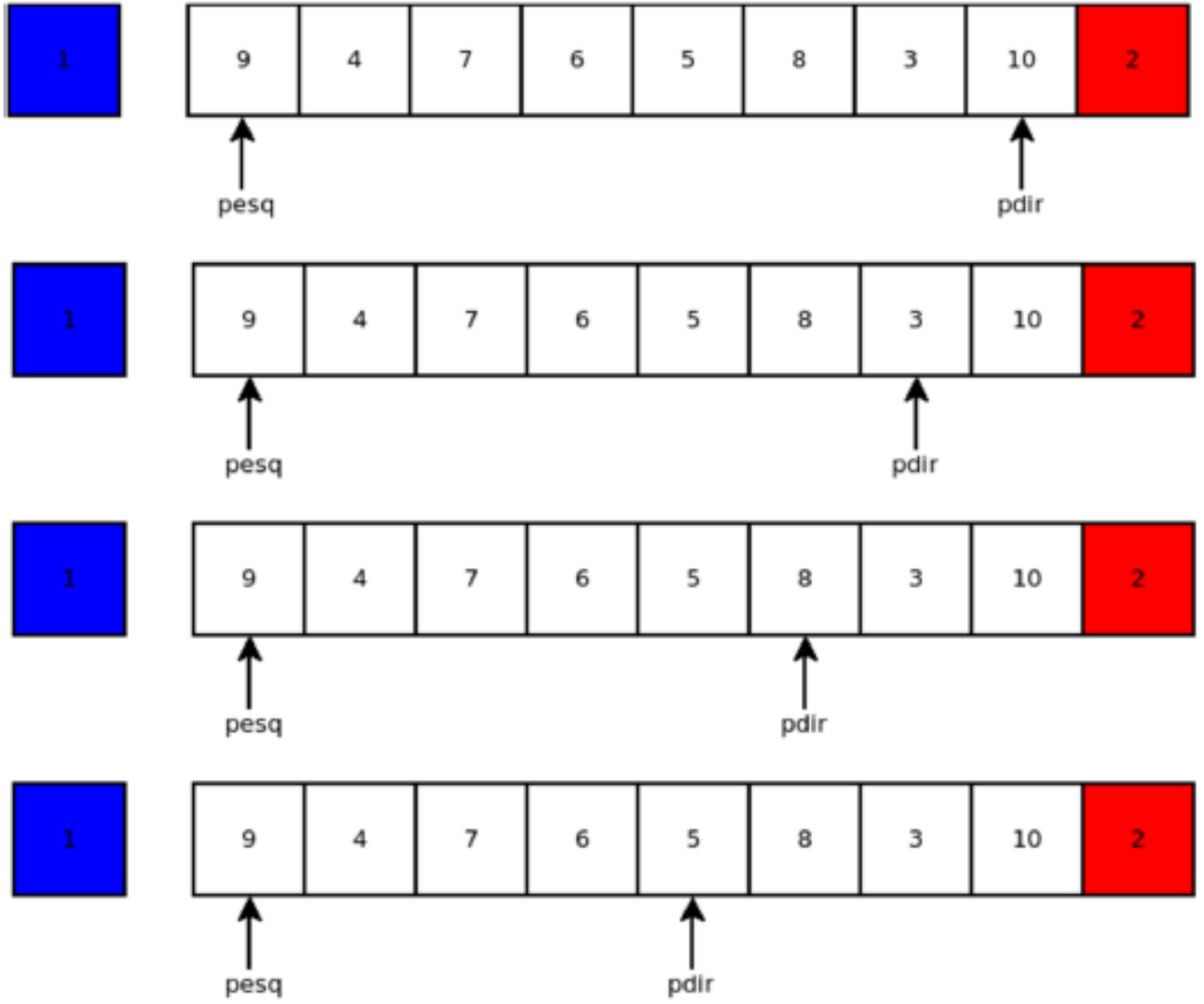
Quick Sort


2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir
2 9 4 7 6 5 8 3 10 1
pesq
pdir

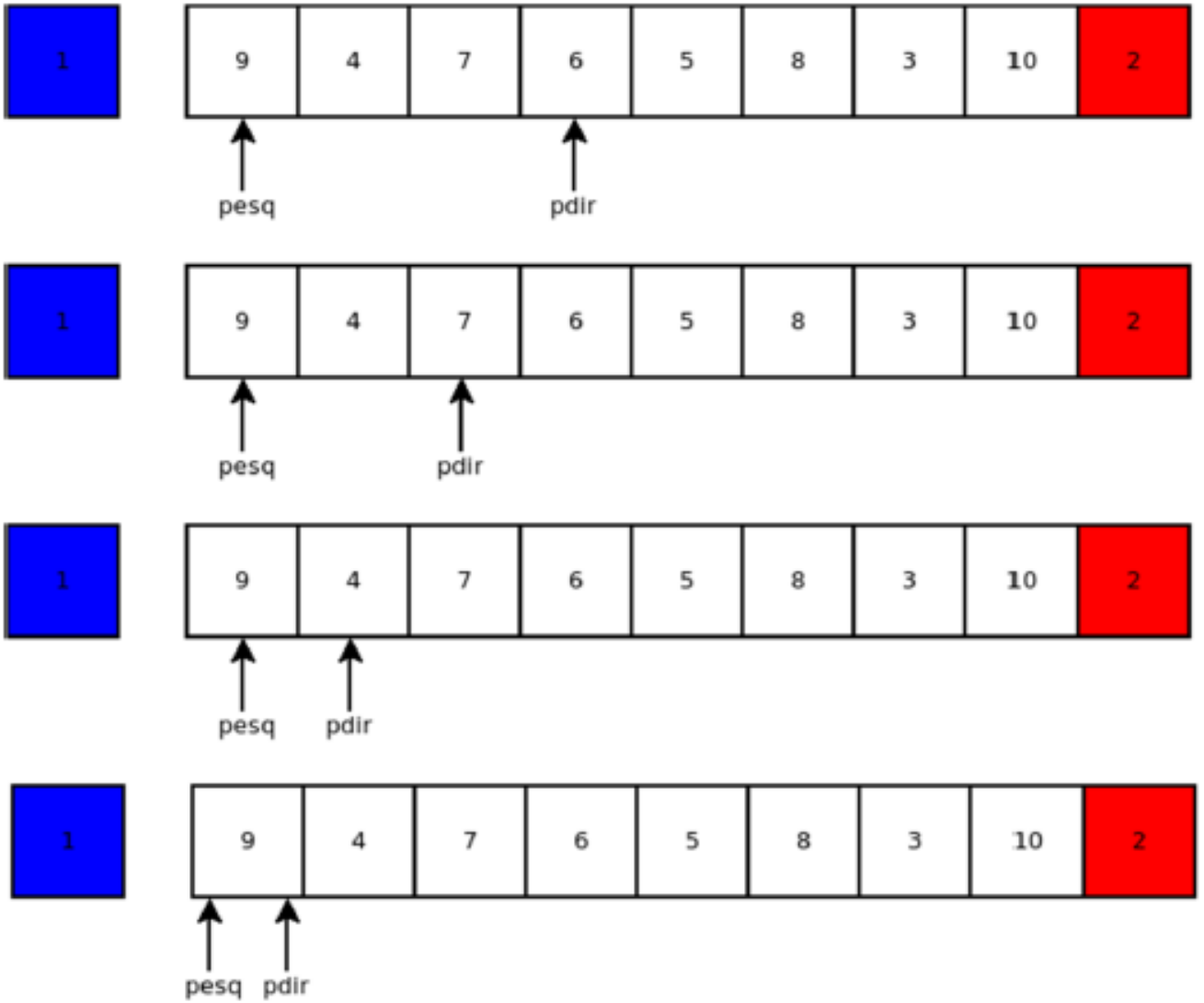
Quick Sort


2 9 4 7 6 5 8 3 10 1
pesq pdir
2 9 4 7 6 5 8 3 10 1
pesq pdir
2 9 4 7 6 5 8 3 10 1
pdir pesq
1
9 4 7 6 5 8 3 10 2

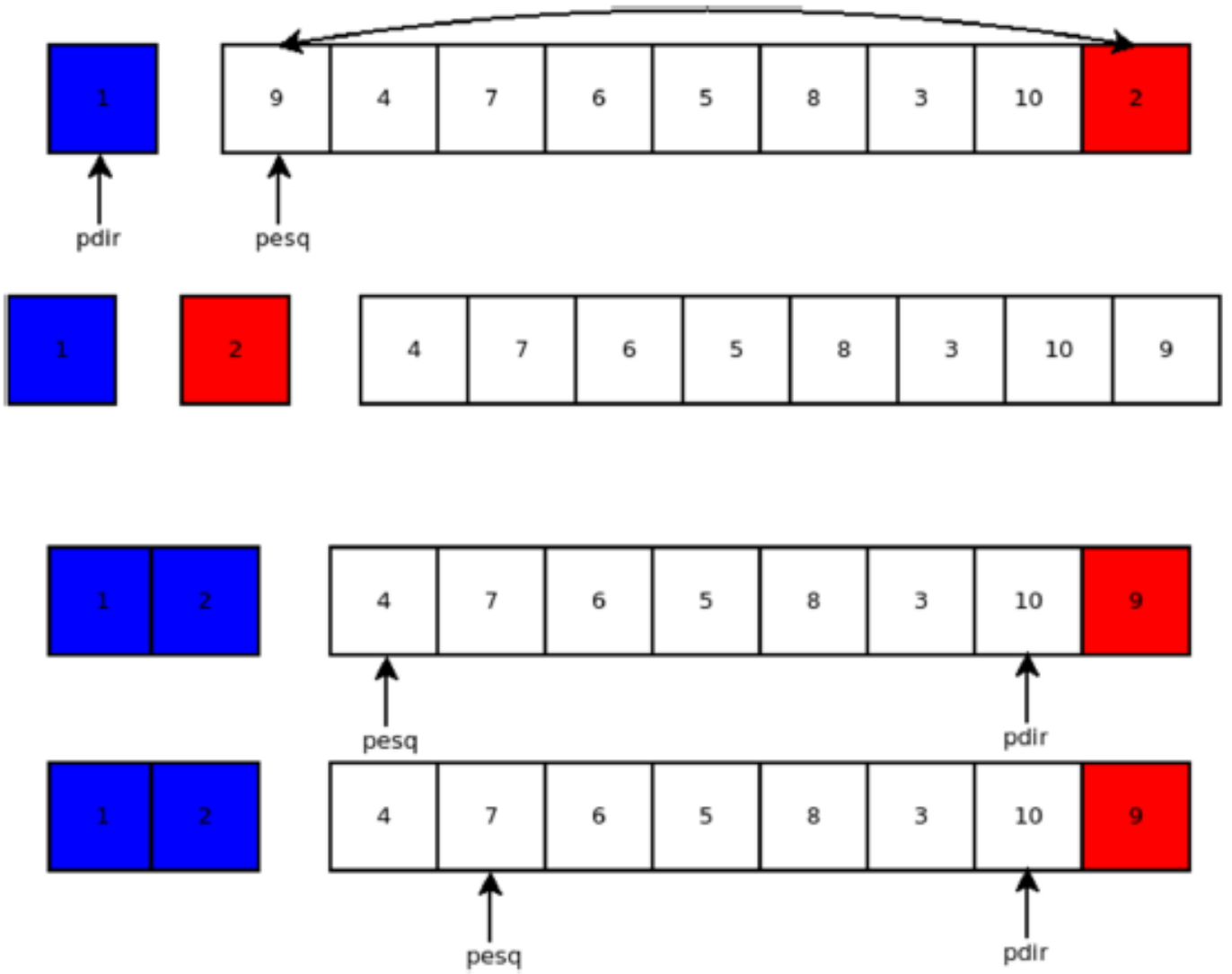
Quick Sort


1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir

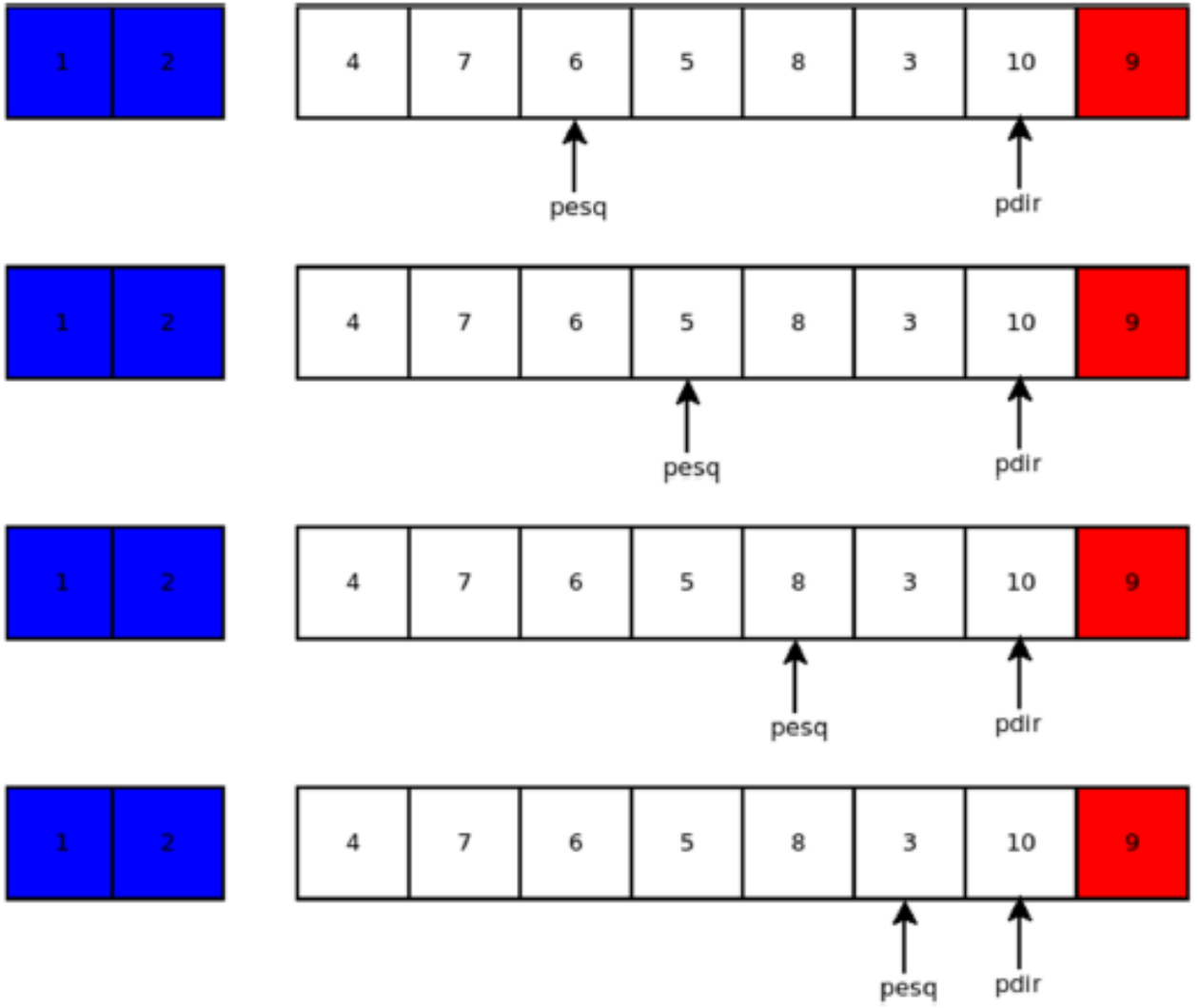
Quick Sort


1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir
1
9
4
7
6
5
8
3
10
2
pesq
pdir

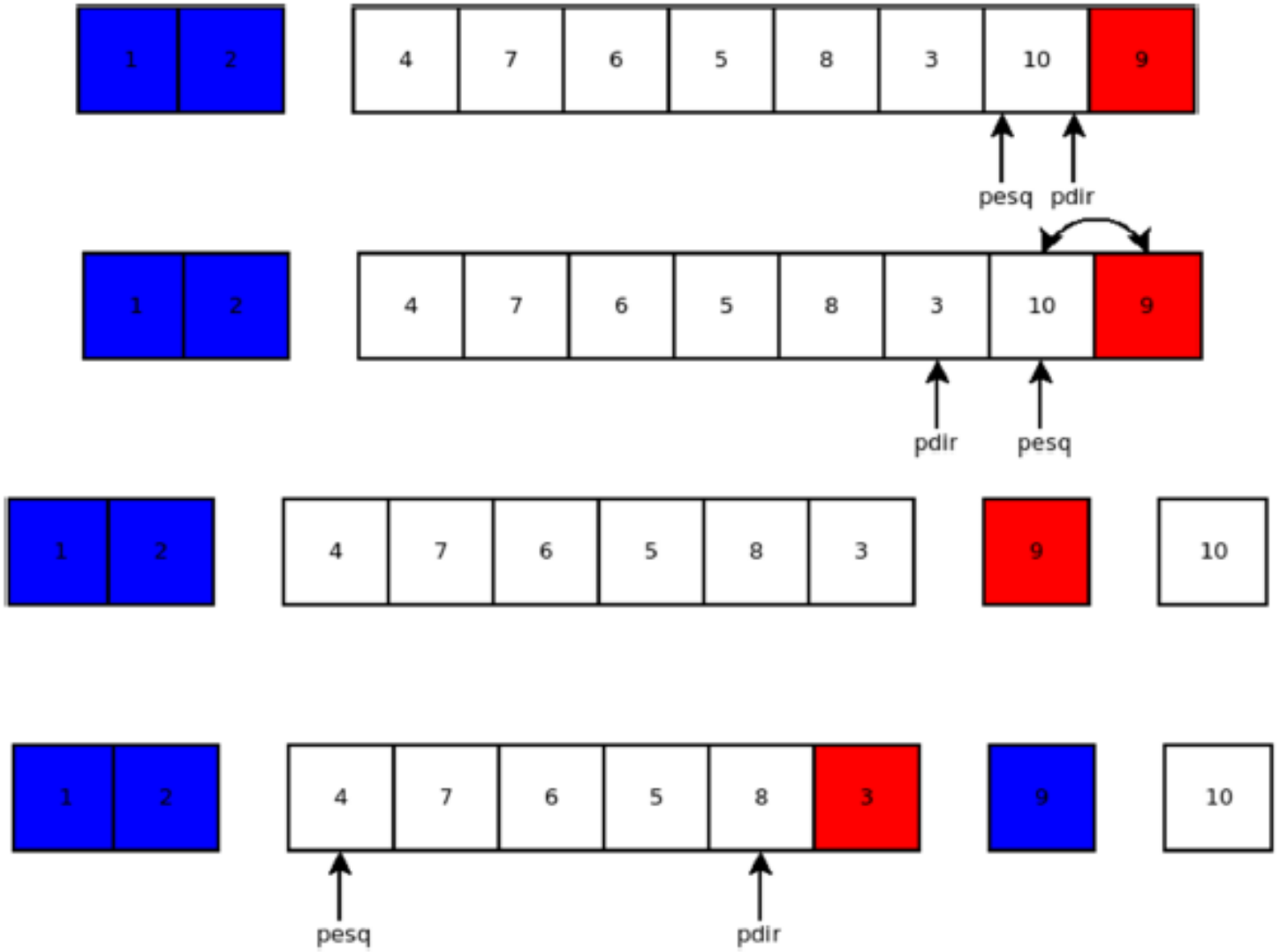
Quick Sort


1
9
4
7
6
5
8
3
10
2
pdir
pesq
1
2
4
7
6
5
8
3
10
9
1
2
4
7
6
5
8
3
10
9
pesq
pdir
1
2
4
7
6
5
8
3
10
9
pesq
pdir

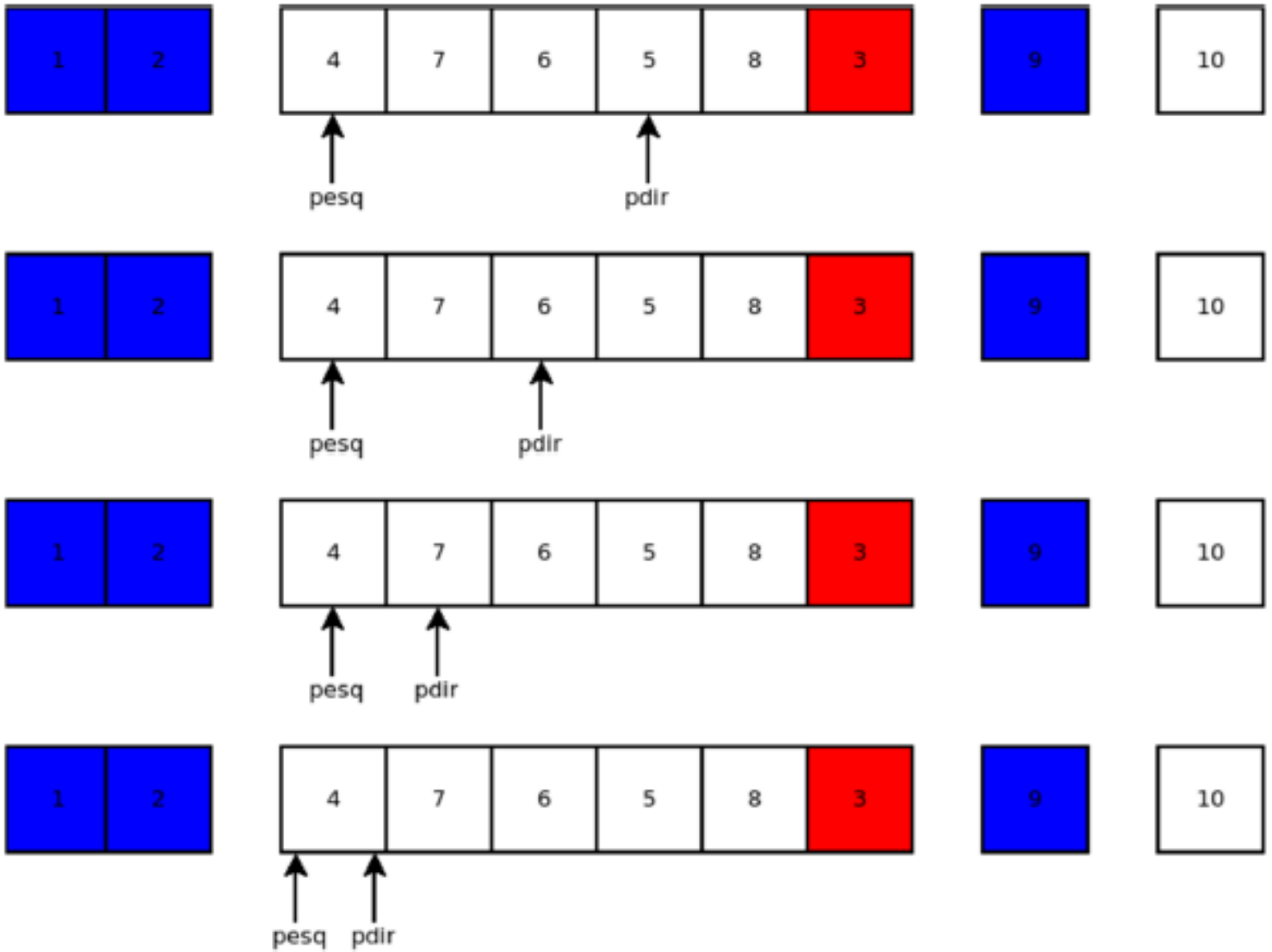
Quick Sort


1
2
4
7
6
5
8
3
10
9
pesq
pdir
1
2
4
7
6
5
8
3
10
9
pesq
pdir
1
2
4
7
6
5
8
3
10
9
pesq
pdir
1
2
4
7
6
5
8
3
10
9
pesq
pdir

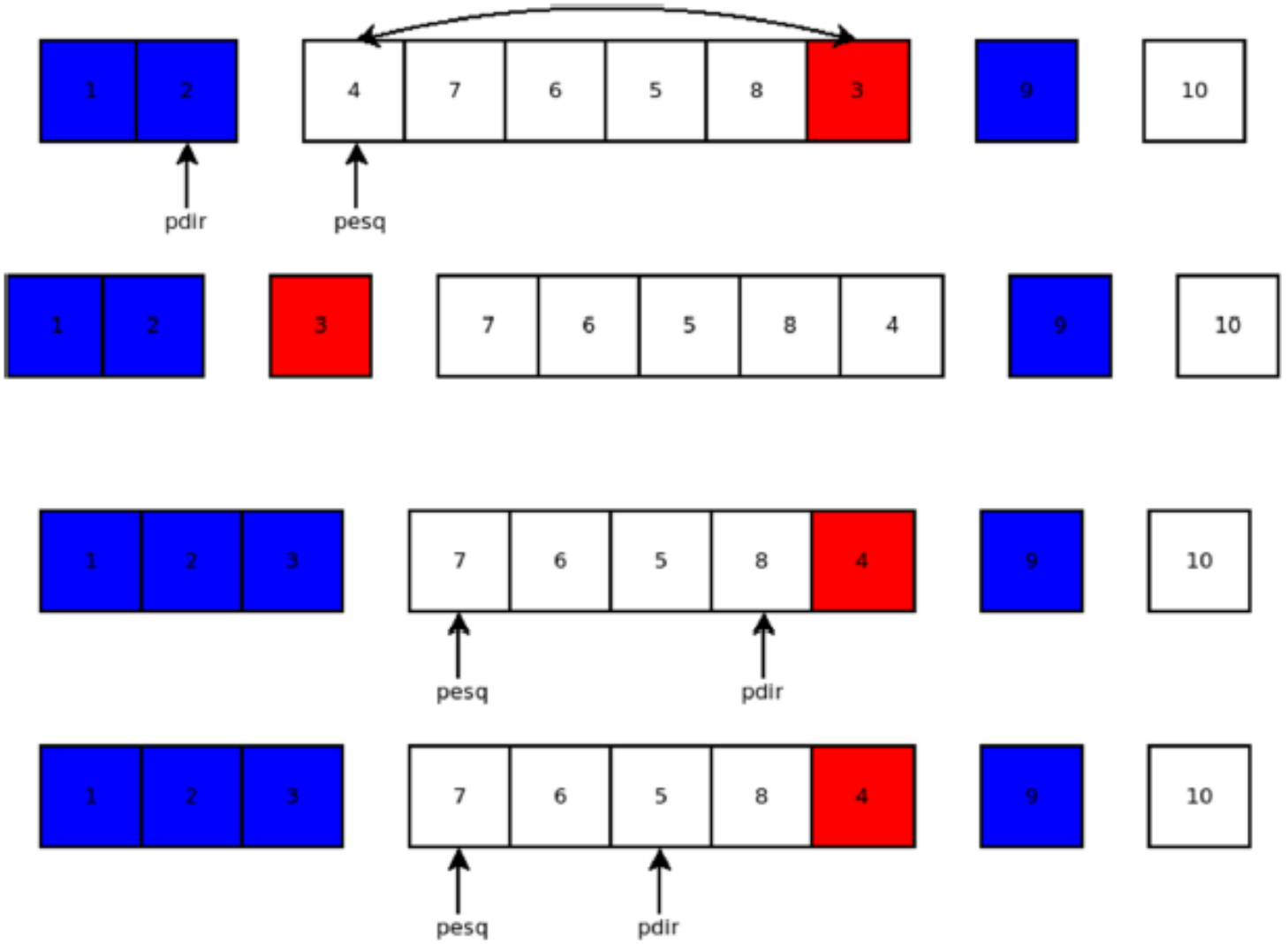
Quick Sort


1
2
4
7
6
5
8
3
10
9
pesq
pdir
1
2
4
7
6
5
8
3
10
9
pdir
pesq
1
2
4
7
6
5
8
3
9
10
1
2
4
7
6
5
8
3
9
10
pesq
pdir

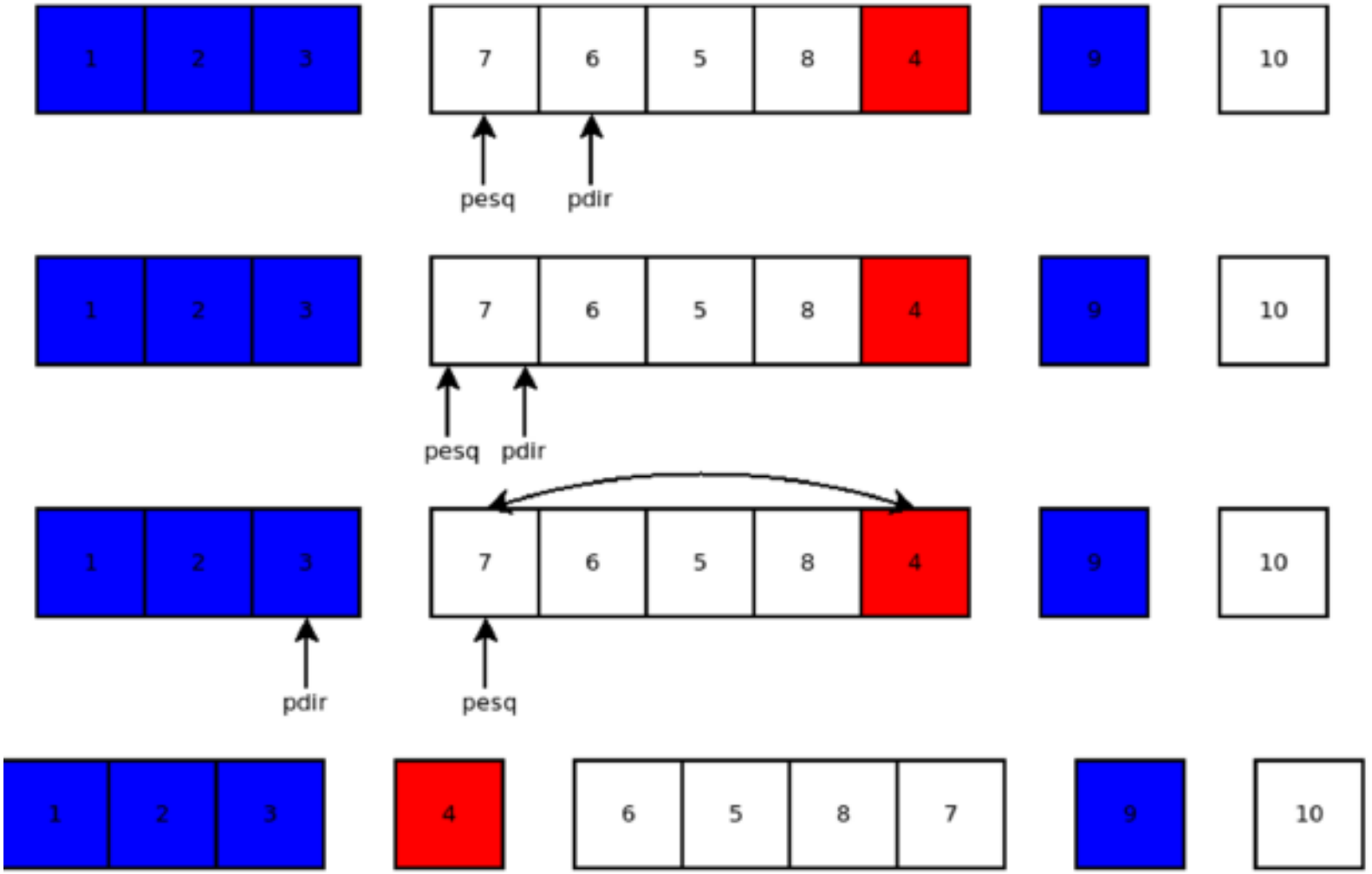
Quick Sort


1
2
4
7
6
5
8
3
9
10
pesq
pdir
1
2
4
7
6
5
8
3
9
10
pesq
pdir
1
2
4
7
6
5
8
3
9
10
pesq
pdir
1
2
4
7
6
5
8
3
9
10
pesq
pdir

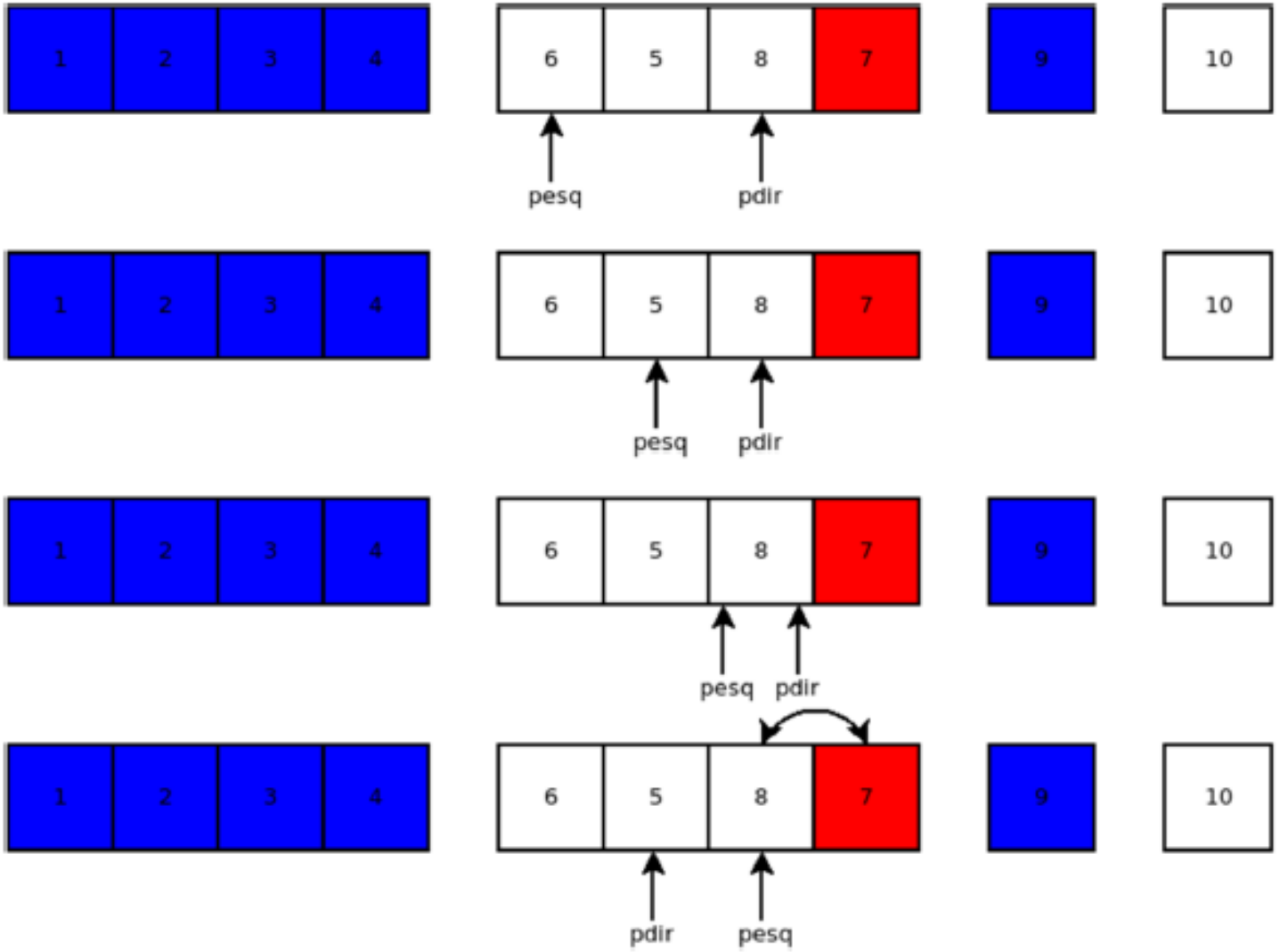
Quick Sort


1
2
4
7
6
5
8
3
9
10
pdir
pesq
1
2
3
7
6
5
8
4
9
10
1
2
3
7
6
5
8
4
9
10
pesq
pdir
1
2
3
7
6
5
8
4
9
10
pesq
pdir

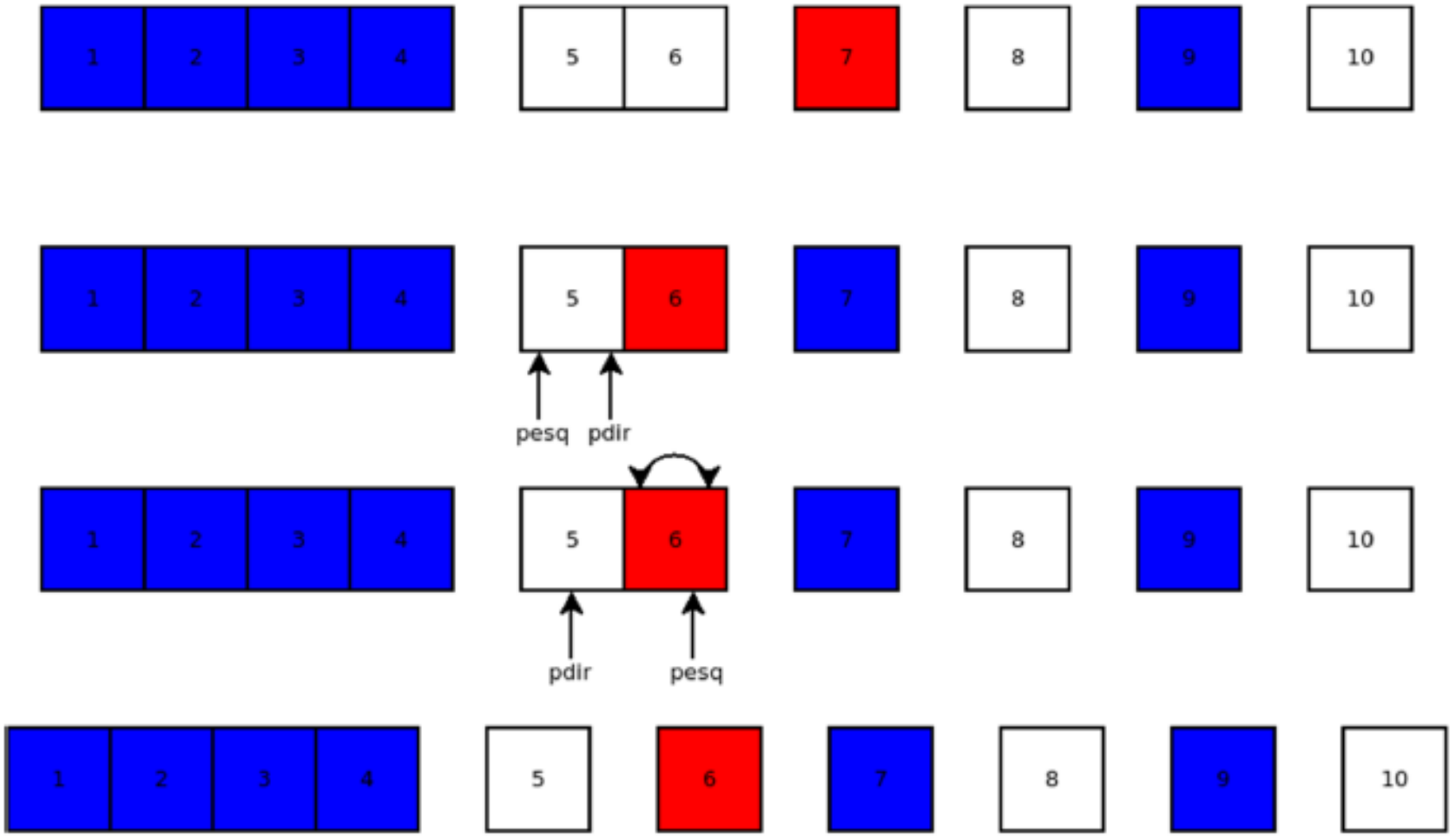
Quick Sort


1
2
3
7
6
5
8
4
9
10
pesq
pdir
1
2
3
7
6
5
8
4
9
10
pesq
pdir
1
2
3
7
6
5
8
4
9
10
pdir
pesq
1
2
3
4
6
5
8
7
9
10

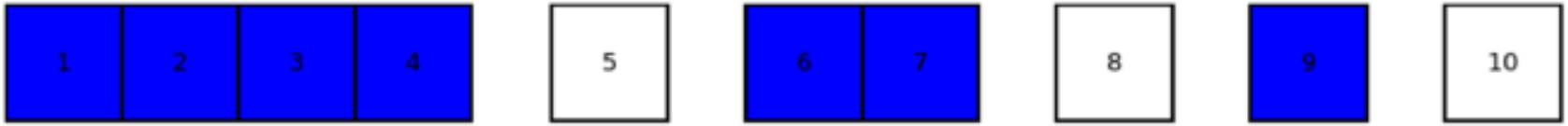
Quick Sort

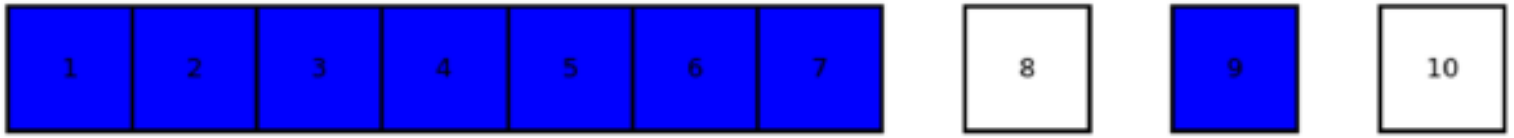

1
2
3
4
6
5
8
7
9
10
pesq
pdir
1
2
3
4
6
5
8
7
9
10
pesq
pdir
1
2
3
4
6
5
8
7
9
10
pesq
pdir
1
2
3
4
6
5
8
7
9
10
pdir
pesq

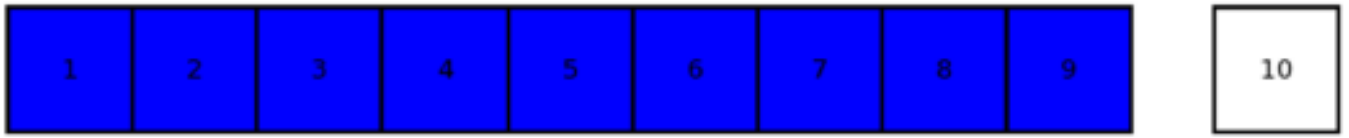
Quick Sort

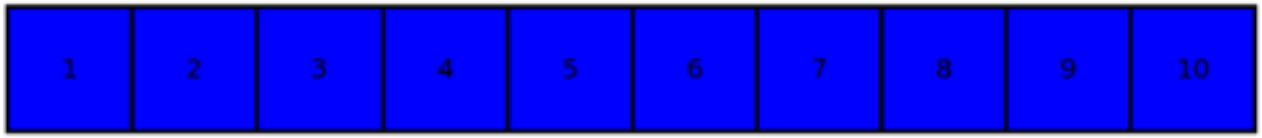


Quick Sort



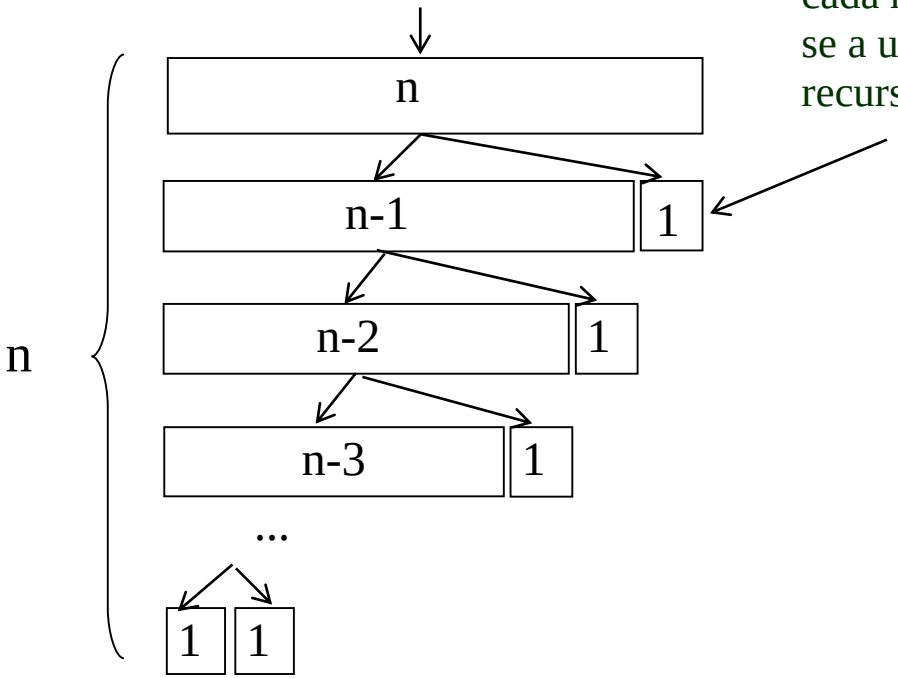






Eficiência do QuickSort

Pior caso:



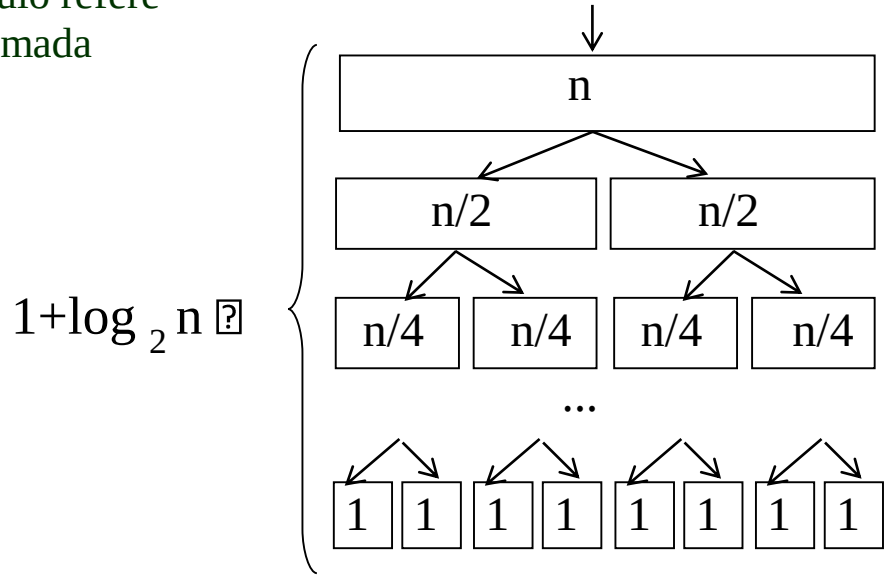
cada rectângulo refere-se a uma chamada recursiva

- profundidade de recursão: *n*
- tempo de execução total (somando totais de linhas):

T(n) = O[n+n+(n-1) + ... +2]

= O[n+(n-1)(n + 2)/2] = **O(n^2)**

Melhor caso:



- profundidade de recursão: *1+log $_2$ n* (sem contar com a possibilidade de um elemento ser excluído dos sub-arrays esquerdo e direito)
- tempo de execução total (uma vez que a soma de cada linha é *n*):

T(n) = O[(1+log$_2$n) n] = **O(n log n)**

Complexidade Espacial **`QuickSort`**

- O espaço de memória exigido por cada chamada de **`QuickSort`**, sem contar com chamadas recursivas, é independente do tamanho (*n*) do vector
- O espaço de memória total exigido pela chamada de **`QuickSort`**, incluindo as chamadas recursivas, é pois proporcional à profundidade de recursão
- ***S(log n)*** no melhor caso (e no caso médio)
- ***S(n)*** no pior caso

QuickSort Híbrido

- Utilizar o quickSort até uma determinada profundidade
- Utilizar outro algortimo para ordenar os últimos elementos.
- Vantagens
- A árvore de recursão é interrompida antes de chegar às folhas

- As chamadas recursivas utilizam espaço na stack do computador e são lentas

Pesquisa Sequencial

Complexidade temporal: O(N)

```
Object[] lista = ....

boolean find(Object o) {
    para i de 0 até lista.tamanho-1
        se lista[i] = o
            Devolve verdadeiro
   devolve falso
}
```

Pesquisa Binária

Complexidade temporal: O(logn)
Limitações: O array tem que estar ordenado

```
Object[] lista = ....


boolean find(Object o) {
    esquerda = 0
    direita = lista.tamanho - 1
    enquanto esquerda <= direita
        meio = (esquerda+direita) / 2
        se lista[meio] < o
            Esquerda = meio + 1
        senao se lista[meio] > o
            direita = meio - 1
        Senao
            devolve verdadeiro
    devolve falso
}
```

Pesquisa Probabilistica

Complexidade temporal: O(n)

Sempre que um valor é pesquisado ele sobe uma posição no array, apesar de teoricamente o algoritmo ser O(n), na prática para os valores que são muitos procurados, a complexidade aproxima-se de O(1)

```
Object[] lista = ....


boolean find(Object o) {
    para i de 0 até lista.tamanho-1
        se lista[i] = o
            Se i > 0
                troca(i, i-1)
            Devolve verdadeiro
   devolve falso
}
```